

Act of Kindness — Donation Portal

Tech Stack, Architecture & Services Reference — for the development team

1. Tech Stack

Layer	Technology	Notes
Frontend	React	Admin dashboard + public donation flow
Backend / API	Node.js + Express	REST API, webhook handlers
Database	PostgreSQL	Single source of truth for all records
Hosting (compute)	AWS ECS Fargate	Runs the Express API in containers
Hosting (frontend)	AWS S3 + CloudFront	Static React build, served via CDN
Payments	Stripe, PayPal	Online donation processing
Bank sync	Plaid (Barclays)	Read-only bank transaction feed
File storage	AWS S3	Receipts / payment proof uploads
Test environment	KVM VM + Docker Compose	Local staging before AWS deploy

2. Architecture Overview

The system is split into a public-facing layer and a fully private layer. Nothing outside the AWS VPC can reach the API or database directly — the load balancer is the only public door.

Step	Component	Purpose
1	CloudFront + S3	Serves the React frontend to donors and admins over HTTPS
2	ALB + WAF (public subnet)	Single public entry point for API traffic; blocks common attacks; TLS termination
3	ECS Fargate (private subnet)	Runs the Express API; not reachable from the internet directly
4	RDS PostgreSQL (private subnet)	Encrypted database; only the API can connect to it
5	Secrets Manager	Stores Stripe/PayPal/Plaid keys and DB credentials; API pulls at runtime
6	S3 (private bucket)	Stores receipts and payment-proof uploads, accessed via signed URLs only

3. AWS Services — Where Each Is Used

Service	Used for	Used in / by
VPC	Isolated network boundary for all infrastructure	Foundation — set up before any other AWS resource

Service	Used for	Used in / by
ECS Fargate	Runs the Express API server	All backend endpoints: donations, campaigns, donors, reports, auth
RDS (PostgreSQL)	Primary database	All modules — donations, donors, campaigns, allocations, audit logs
S3 (frontend bucket)	Hosts the built React app	Public site + admin dashboard
S3 (uploads bucket)	Stores receipts / offline payment proof	Offline Donations module (Section 9 of scope doc)
CloudFront	CDN + HTTPS delivery for the frontend	Every page load
ALB	Routes and load-balances API traffic	Every API request
AWS WAF	Blocks common web attacks (SQLi, bots)	Attached to the ALB
Secrets Manager	Secure storage for API keys and credentials	Stripe, PayPal, Plaid, and DB credentials
ACM	Free TLS certificates	CloudFront + ALB, enforces HTTPS everywhere
Route 53	DNS for the domain	Points the domain at CloudFront / ALB
IAM Roles	Least-privilege permissions per service	ECS tasks, Lambda (if used), backup jobs
CloudTrail	Logs every action taken in the AWS account	Infrastructure-level audit trail
GuardDuty	Threat detection	Account-wide monitoring
CloudWatch	Application logs and alarms	API logs, failed-login alerts, DB load alerts

4. Third-Party Integrations

Service	Purpose	Used in
Stripe	Online card payments	Website donation flow, Payment Transactions
PayPal	Alternate online payment method	Website donation flow, Payment Transactions
Plaid	Read-only sync of Barclays bank transactions	Bank feed reconciliation, Offline Donations

5. Security Measures (non-negotiable, built into every phase)

- Webhook signature verification on every Stripe, PayPal, and Plaid callback
- Plaid access_token and all API keys encrypted / stored only in Secrets Manager, never in code
- Role-based access control (Super Admin / Admin / Manager / Volunteer) enforced server-side, not just hidden in the UI
- Audit log on every change to a financial record: who, what, old value, new value, when

- Database and API isolated in private subnets — never directly reachable from the internet
- TLS/HTTPS enforced everywhere (ACM certificates, no plain HTTP)
- Password hashing, rate limiting, and secure file upload validation on receipt/proof uploads
- Automated RDS backups + S3 versioning as a last line of defense

This document reflects the architecture agreed for the AOK Donation Portal build. Refer to the full developer scope brief for detailed functional requirements per module.